

Laboratório 6 — “Não tem como ganhar todas.... será mesmo?”

Esse lab vai ser um pouco diferente: nada de grandes explicações ou novos conceitos por agora, vamos direto colocar a mão na massa! (ou no teclado...)

Abaixo estão alguns trechos de código que vão te ajudar na implementação do ambiente Blackjack, mas com Q-Learning dessa vez.

Python

```
'''plotar o gráfico da porcentagem de recompensas (episódios)'''

win_rate = np.zeros(episodes)

for t in range(episodes):

    window = rewards_per_episode[max(0, t-100):(t+1)]

    win_rate[t] = np.mean(window) * 100

plt.plot(win_rate)

plt.xlabel("Episodes")

plt.ylabel("Episode Rewards")

plt.savefig('blackjack.png')
```

Python

```
'''

plotar a tabela com a visualização da política

'''

import numpy as np

import matplotlib.pyplot as plt

from matplotlib.patches import Patch

import matplotlib.colors as mcolors

def build_policy_array(q, usable_ace=False, hit_until=20):

    """

    retorna (grid, players, dealers) para plot:
```

```
- grid: matriz 10x10 com 0 (S) ou 1 (H)
- players: [12..21] (eixo X)
- dealers: [1..10] onde 1 = Ás (eixo Y)

"""

dealers = np.arange(1, 11)    # 1 = Ás
players = np.arange(12, 22)   # 12..21
grid = np.zeros((len(dealers), len(players)), dtype=int)

for i, d in enumerate(dealers):
    for j, p in enumerate(players):
        s = (int(p), int(d), bool(usable_ace))

        if s in q:
            a = int(np.argmax(q[s]))    # 0=S, 1=H
        else:
            a = 1 if p < hit_until else 0

        grid[i, j] = a

return grid, players, dealers

def plot_policy_matplotlib(grid, players, dealers, title='Policy', savepath=None):
    """
    plota a política como heatmap 2D.

    - nnota 'H'/'S' em cada célula.

    - eixos: X = soma do jogador (12..21), Y = carta visível do dealer (A,2..10)
    """

    fig, ax = plt.subplots(figsize=(8, 6))

    # mapa de cores : 0 = cinza (stick), 1 = verde (hit)
    cmap = mcolors.ListedColormap(['lightgrey', 'lightgreen'])

    bounds = [-0.5, 0.5, 1.5] # fronteiras entre categorias

    norm = mcolors.BoundaryNorm(bounds, cmap.N)

    # mostrar a matriz
```

```
im = ax.imshow(grid, origin='upper', aspect='equal', cmap=cmap, norm=norm) # 0..1

# ticks e rótulos
ax.set_xticks(np.arange(len(players)))
ax.set_yticks(np.arange(len(dealers)))
ax.set_xticklabels(players)
ax.set_yticklabels(['A'] + list(map(str, range(2, 11))))

# linhas de grade leves
ax.set_xticks(np.arange(-0.5, len(players), 1), minor=True)
ax.set_yticks(np.arange(-0.5, len(dealers), 1), minor=True)
ax.grid(which='minor')
ax.tick_params(which='minor', bottom=False, left=False)

# anotações H/S nas células
for i in range(len(dealers)):
    for j in range(len(players)):
        txt = 'H' if grid[i, j] == 1 else 'S'
        ax.text(j, i, txt, ha='center', va='center')
ax.set_xlabel('Player sum')
ax.set_ylabel('Dealer showing')
ax.set_title(title)

# legenda
legend_elements = [
    Patch(facecolor="lightgreen", edgecolor="black", label="Hit"),
    Patch(facecolor="lightgrey", edgecolor="black", label="Stick"),
]
ax.legend(handles=legend_elements, bbox_to_anchor=(1.3, 1))
plt.tight_layout()

if savepath:
    plt.savefig(savepath, dpi=150)
```

```
plt.show()

# ás usável como 11
grid_ua, players, dealers = build_policy_array(q, usable_ace=True)
plot_policy_matplotlib(grid_ua, players, dealers,
                        title='Policy (usable ace = True)',
                        savepath='policy_usable_ace.png')

# ás não usável como 11
grid_nua, players, dealers = build_policy_array(q, usable_ace=False)
plot_policy_matplotlib(grid_nua, players, dealers,
                        title='Policy (usable ace = False)',
                        savepath='policy_no_usable_ace.png')
```

Depois de implementar o ambiente e visualizar como se deu o aprendizado, podemos pensar: existem diferenças entre o comportamento das curvas de recompensa do Frozen Lake e do Blackjack? Se sim, quais são elas e qual pode ser uma explicação possível?

Mais informações

Implementação do Blackjack usando Q-Learning segundo a biblioteca oficial do Gymnasium:

https://gymnasium.farama.org/tutorials/training_agents/blackjack_q_learning/